

DEEPTrip TECH TALK

AI Agent 在 DeepTrip 项目的 研发链路自动化实践

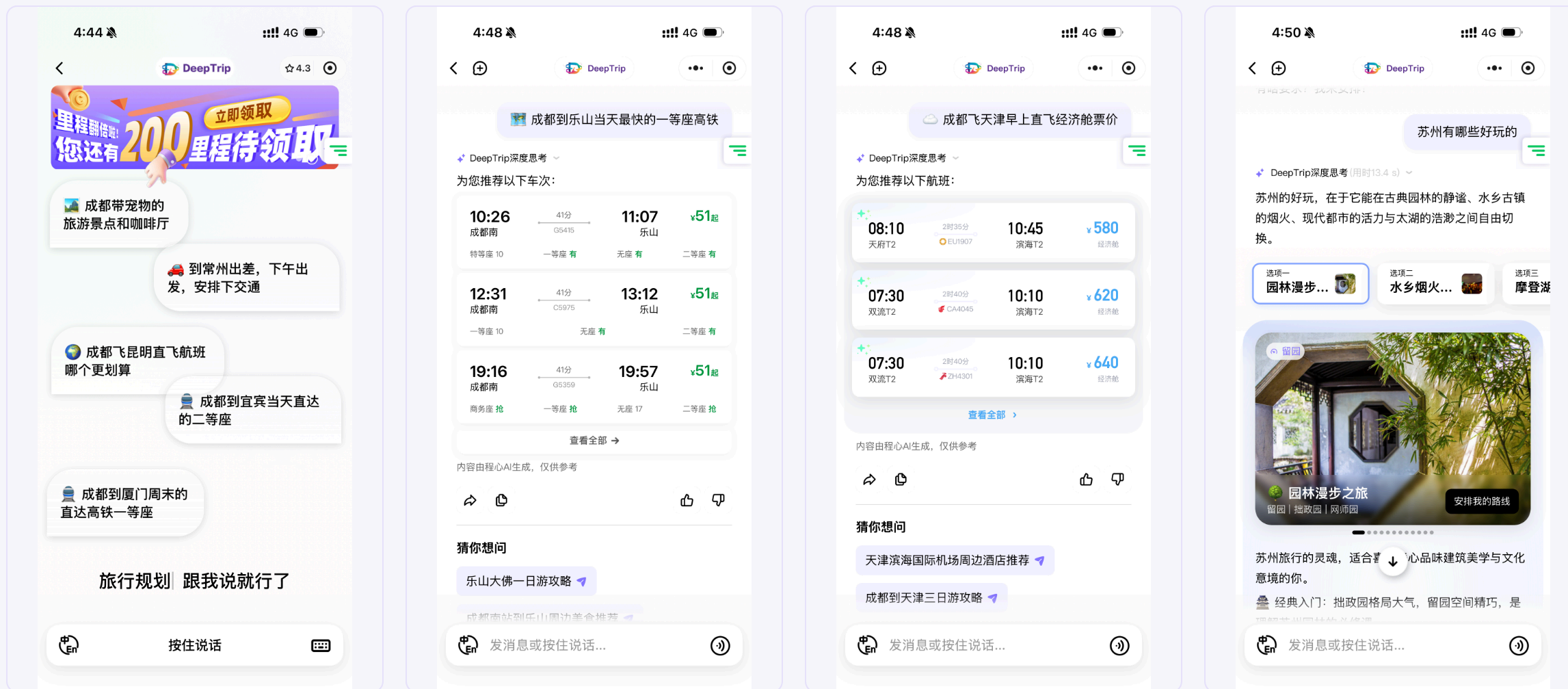
研发模式转变

工程效能提速

产品创造力释放

DeepTrip Team · 徐凯旋 · 2026.04.22

DeepTrip — 旅行类大模型应用产品



致力于成为旅途中最贴心的智能管家

演进之路

从"补全"到"协作"到"自治"——从"写代码"到"做决策"

2021.06

**GitHub
Copilot**

开创 AI 编程先河

2023.01

**Cursor
发布**

AI-first IDE

2025.02

**Claude
Code**

CLI 原生
Agent

2025.06-08

生态爆发

Gemini/Qwen/DeepSeek

2025.11

**GPT-
5.1-Max**

项目级长时程

2026.01

**能力再次
跃升**

GPT-5.3 /
Codex App
视觉编码 +
Swarm

2022.06

**Copilot 正
式版**

订阅上线

2024

**Cursor
Agent**

理解代码库，
多文件编辑

2025.04-05

**Codex
Agent**

并行任务沙箱

2025.09

**GPT-5-
Codex**

长上下文突破

**2025.12 —
模型密集发布
期**

GPT-5.2-Codex
长时程一致性 + 工具调用

Claude Opus 4.5
SWE-bench 领先，定价降 67%

DeepSeek-V3.2
长时程一致性质变

Kimi K2.5 视觉编
码 + Agent
Swarm

MiniMax M2.5
SWE-bench 80.2%

GLM-5 长时程

**2026.02-03
— 能力再次跃
升**

GPT-5.3-Codex
视觉编码 +
Swarm

Claude Opus 4.6
1M 上下文 +
Agent Teams

**Claude Sonnet
4.6** 日常 coding
最优平衡

**Qwen3.5 /
Qwen3-Coder-
Next** 本地/小模型
agent

Gemini 3 系列 高
推理 agent 基准领

我们正处于 **第三次范式跃迁** 的起点——AI 从"辅助"变成"队友"

dt_robot 诞生记 — AI Coding 浪潮下的必然选择

01 LLM Coding 快速发展

Copilot → Cursor → Claude Code
→ Codex
AI 编程从"补全"进化到"自治"
开发范式在发生根本性变化

02 能力介入开发全流程

AI 不再只写代码
开始 **理解项目**、**执行流程**、**做决策**
从工具变成队友

03 Claude Code 自己开发自己

一个标志性事件
用自己的产品来开发自己的产品

04 DeepTrip 需要统一的开发体系

加速创新类产品的 **开发速度**
让项目组全部使用 **同一套体系**
为产品带来更多可能性

05 dt_robot 掌握 DeepTrip 的一切

开发提速 · **业务知识收敛** · **生产数据随时问** · **生产环境及时修复**
.....还有更多

04 dt_robot 到底是什么 — 一个仓库，一种工作方式

- 01

文档

需求 + 方案 + 迭代记录，代码旁随时可读
- 02

约束

告诉 AI 该怎么做、什么不能做
- 03

技能

30 个标准化 SOP，覆盖开发全流程
- 04

流程

10 阶段迭代：需求 → 设计 → 编码 → 评审 → 交付
- 05

脚本

一键启动、检查、测试、完成通知

~/vibe_coding/arsenal-service-ai-dt-robot

git main *1 !3

username: gcp

coglm5t_noask_may_danger

切换到 OneAI GLM-5-Turbo 模型 ...

已切换到 OneAI GLM-5-Turbo 模型

BASE_URL: https://oneai.17uoft.com/anthropic

MODEL: glm-5-turbo

Claude Code v2.1.97

glm-5-turbo · API Usage Billing

~/vibe_coding/arsenal-service-ai-dt-robot

nginx -d pronunciation-game.com -d

hi,介绍下deeptrip这个产品和dt_robot,

●DeepTrip 产品

DeepTrip 是一个行程/旅行智能体产品，围绕 AI 驱动的旅行规划与预订服务。多仓库协作工程，核心仓库包括：

仓库	职责
arsenal-ai-deeptrip	服务端总入口（customer-api 前端 + merchant-api 内部管理）
agent-b101	Agent 核心（编排、工具调用、策略）
arsenal-ai-deeptrip-marketing	营销与活动
arsenal-service-ai-dataset	数据集（火车票/景点/机票/酒店/地图）
arsenal-service-ai-mcp	统一数据管理与对外能力
ai-booking	H5 前端
deeptrip-applet	小程序前端

团队约 8 人、6 个角色。

dt_robot（本仓库）

dt_robot 是为 DeepTrip 服务的开发运营基础设施，不是 DeepTrip 业务本身。主要提供：

- Robot 管理平台 (/admin)：管理用户/角色/权限/文档，需要 PERM_ADMIN
- dt_dev_admin 登录鉴权：通过 auth API 为独立前端项目提供认证
- 文档管理：wiki、迭代文档、项目文档的存储与检索
- 开发流程支撑：迭代管理、代码评审、构建部署等辅助能力
- 分享与 Slides：团队技术分享等静态内容托管

简单说：DeepTrip 是产品，dt_robot 是造产品过程中搭建的工具和平台。

05 我是怎么做的（一）——梳理文档体系

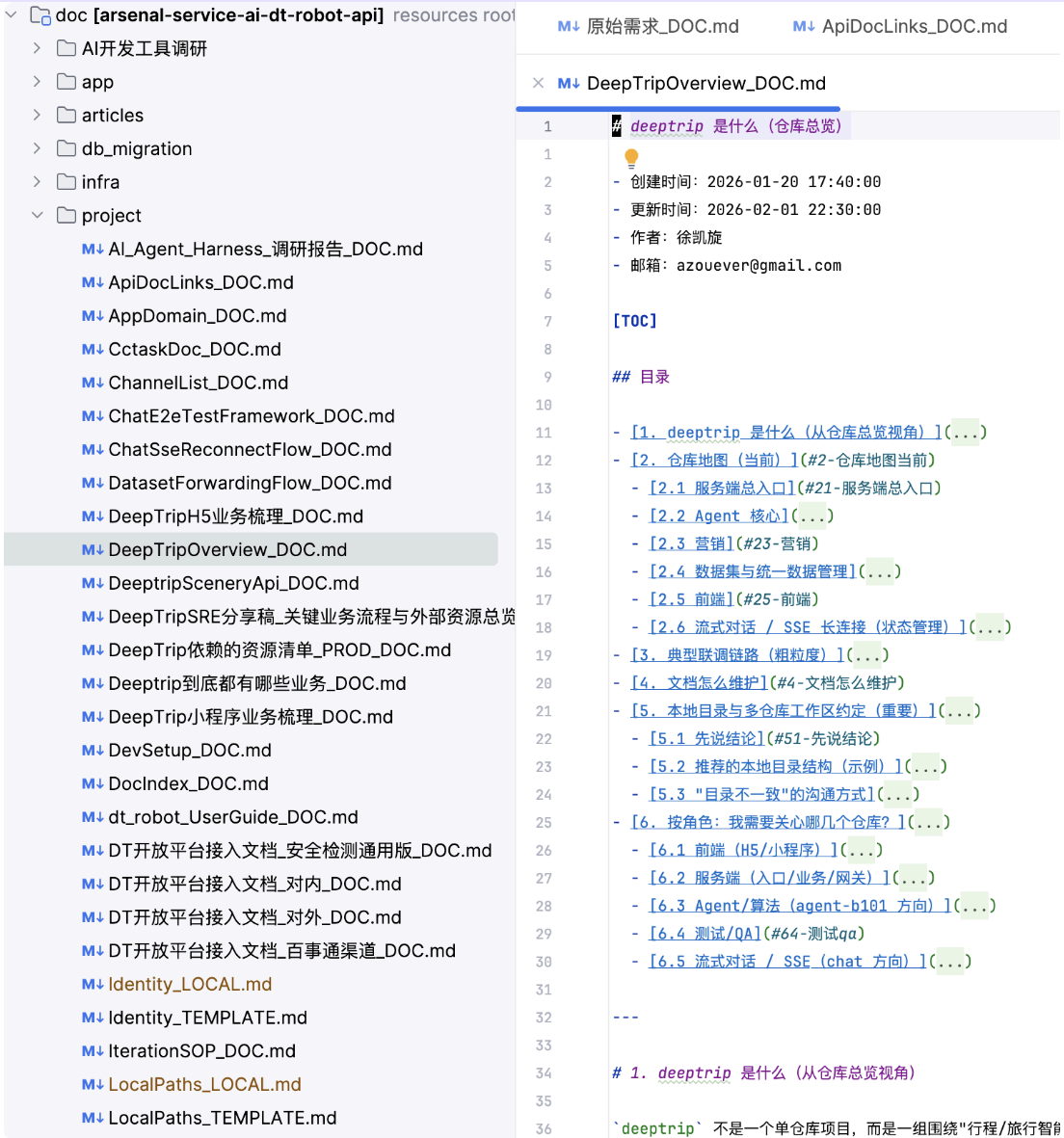
文档体系

项目文档

业务文档

技术文档

开发流程



05 我是怎么做的（一）——文档中心

应用筛选:

Robot (本地)

服务端总入口

Agent核心

数据集

营销

MCP数据能力

全选

清空

环境选择:

QA (默认)

QA1

QA2

QA3

QA4

QA5

QA6

QA7

QA8

QA9

全选

清空

原始

☒ 摘要

最近更新

按名称

ESC 清空

📦 AI开发工具调研

1 篇

📦 articles

3 篇

Claude Code 最佳实践 (GitHub 社区版)

新标签

原始 md

articles/ClaudeCode最佳实践

2026-04-14 09:54

... R 代码分析 (95% Token 节省) > 来源: parcadei/Continuous-Claude-v3 (3588 stars) 用 5 层抽象替代阅读原始代码: | 层级 | 内容 | 增加 Token ||:---|:---|:---| | L1: AST | 函数、类、签名 | ~500 | | L2: C ...

DeepTrip 超级动力引擎: dt_robot

新标签

原始 md

articles/DT_Robot_团队分享

2026-04-14 09:54

... Claude Code 会自己做的事: 1. 抓取 PRD 内容 2. 触发 requirement-analysis skill 3. 建迭代目录 4. 写 原始需求_DOC.md (产品原文存档 + AI 整理版) 5. 写 需求拆解_DOC.md (场景/规则/验收/边界) 6. 写 README.md (迭代看板) 7. ...

Claude Code 实战技巧大全

新标签

原始 md

articles/ClaudeCode实战技巧大全

2026-04-14 09:54

... 务用不同会话。### 21. 用 /fork 分叉实验 想试试不同的方案? ```text /fork experiment-a ``` 创建当前对话的分支。原始会话不受影响。 --- ## 四、终端与输入优化 ### 22. 配置 Option 键为 Meta macOS 终端的 `Alt` 快捷键默认不工作, 需要配置 ...

📦 项目与架构说明

5 篇

同程旅行 DT 开放平台接入文档 (对外)

新标签

原始 md

project/open-platform/DT开放平台接入文档_对外

2026-04-14 09:54

... 字符串 4. 使用 `secret\_key` 和 RSA 算法计算签名 5. 对签名结果进行 Base64 编码 **批量接口示例: ** ```json // 原始批量请求体 { "items": [{ "q": "参考如下对话 [q:'你好',a:'你好,你想去哪玩?';'q':'苏州,2天']", "loc": { " ...

06 我是怎么做的（二） — 梳理产品开发流程

需求分析 > 技术方案 > 编码 > 测试

- 系统迭代
 - 20260205_文档中心与权限系统
 - 20260206_网络活动数据查询
 - 20260207_DeepTrip统计与监控
 - 20260228_任务中心
 - 20260310_任务中心V2
 - 20260310_演示稿系统
 - 20260316_Error日志收集与管理
 - 20260324_自主Agent系统
 - 20260325_Agent日志治理
 - deeptrip外部http资源依赖总览_DOC.md
 - README_DOC.md
- 需求迭代
 - 20260103_江苏省景区实况功能
 - 20260226_程心AI-Haipay开发对接
 - 20260303_Chat_WebSocket对话通道
 - 20260310_地面交通工具接入
 - 20260312_机票业务闭环
 - 20260318_多语言卡片翻译适配
 - 20260327_场景输入配置服务化
- 20260407_用户长期记忆
 - README.md
 - 原始需求_DOC.md
 - 工时评估_DOC.md
 - 技术方案_DOC.md
 - 编码实施计划_DOC.md
 - 需求拆解_DOC.md
- DeepTrip需求开发规范_DOC.md

07 我是怎么做的（三） — 从 CLAUDE.md 到 Skill

V0 只有 CLAUDE.md

所有约束写在一个文件
AI 看了就忘，每次重新说
说漏了就不做

V1 Commands

斜杠调用的可复用 workflow
不用每次复制粘贴长提示
/commit /review 一次搞定

V2 Skills 1.0

加目录 + 支撑文件 + 按需加载
AI 自动检测任务激活技能
2025.10 发布，开放标准

coding-standards 说一句"按代码规范来"，7 维度全覆盖

- 1 定位 只要碰代码就按这个来，写/改/重构/评审全生效
- 2 7 维度总览
 - 通用编码
 - 外部调用
 - 数据层
 - 可观测性
 - 异常处理
 - 性能并发
 - 安全编码
- 3 自检清单 基础项 + 外部调用 + 数据层 + 质量兜底，提交前逐条过
- 4 references/ 按需加载 涉及 DB 才读数据层，涉及外部调用才读外部调用
- 5 语言专项 不同语言可新增专项规则，通用规范全语言适用
- 6 协作链 system-design 设计 → coding-standards 落地 → code-review 检查

skills

.system

build

chat-smoke-test

code-review

code-write-doc

coding-standards

references

M1-01-通用编码规范.md

M1-02-外部调用规范.md

M1-03-数据层规范.md

M1-04-可观测性规范.md

M1-05-异常与错误处理规范.md

M1-06-性能与并发规范.md

M1-07-安全编码规范.md

M1-08-测试规范.md

SKILL.md

constitution

dataset-local-e2e-test

dev-handoff

evaluator

git-commit

git-worktree-starter

integration-test

langfuse-session-query

local-agent-e2e-test

local-dt-main-e2e-test

md-optimizer

mermaid-visualizer

plan-work

pua-debugging

refactor

requirement-analysis

send-email

system-design

system-maintenance

table-design

test-driven-development

coding-standards（编码总规范）

[TOC]

目录

- 定位
- 何时生效
- 规范维度总览
- 编码自检清单
- 语言专项
- 详细规范索引
- 与其他 Skill 的关系

定位

只要碰代码，就按这个来。不管是写新代码、改老代码、还是重构，所有代码最终都要符合这套标准。

不用每次说“注意日志”“注意性能”“注意异常处理”——说一句“按代码规范来”或“按 coding-standards 检查”，以下全部维度自动覆盖。

何时生效

- 写新接口/新功能时
- 修改现有代码时（改到的地方按规范来）
- 重构老代码时（refactor 管怎么拆，本规范管拆完的代码质量）
- 代码评审时（code-review 的检查依据就是本规范）
- 哥说“按代码规范来”“检查一下代码”“规范一下”

规范维度总览

#	维度	核心关注	详细规范
1	通用编码	命名、结构、注释、文件/方法粒度、分库解耦与模块化	references/01-通用编码规范.md
2	外部调用	Bound Service、超时、重试、降级、耗时日志、存储 SDK 网络规范	references/02-外部调用规范.md
3	数据层	DB/缓存/MQ/ES/MongoDB/文件存储	references/03-数据层规范.md
4	可观测性	日志、traced、耗时分桶、脱敏、监控	references/04-可观测性规范.md
5	异常与错误处理	异常体系、错误码、降级、重试策略	references/05-异常与错误处理规范.md

07 我是怎么做的（三） — Skill 编写调教

流 流程层

需求分析 → 技术方案 → 编码
规划 → 编码 → 评估 → 评审
→ 交付 → 编译 → 测试

范 规范层

编码规范（8 维度）· 代码评审
· 代码重构 · 测试驱动开发

操 操作层

数据库表设计 · 文档同步 · Git
提交 · Worktree 并行开发

撑 支撑层

工作规范 · Markdown 优化 ·
Mermaid 画图 · Skill 质量评估

20+ Skills, 一版版迭代出来, 不是一次写好 — 调教到相对稳定态

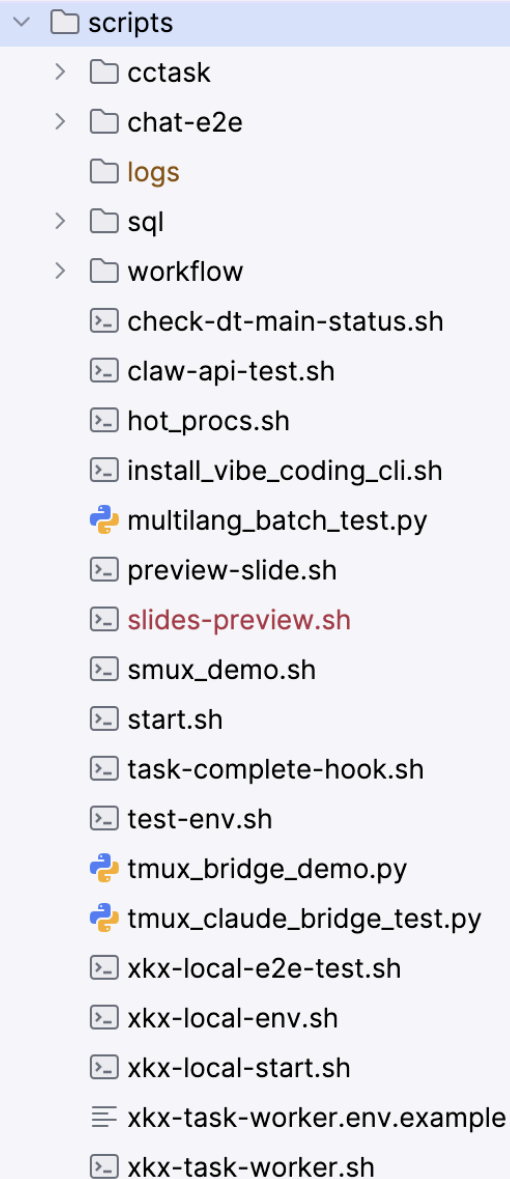
```

  skills
  > .system
  > autonomous-development
    SKILL.md
  > chat-smoke-test
  > code-review
  > code-write-doc
  > coding-standards
  > constitution
  > dataset-local-e2e-test
  > dev-handoff
  > evaluator
  > git-commit
  > git-worktree-starter
  > integration-test
  > langfuse-session-query
  > local-agent-e2e-test
  > local-dt-main-e2e-test
  > md-optimizer
  > mermaid-visualizer
  > plan-work
  > pua-debugging
  > refactor
  > requirement-analysis
  > send-email
  > skill-judge
  > system-design
  > system-maintenance
  > table-design
  > test-driven-development
  > work-spec
  > workflow
  > xkx-local-e2e-test
```

07 我是怎么做的（三） — 脚本能力

脚本能力

Skill + 脚本，让流程更稳定地跑
标准化流程模板，规范执行路径



```
scripts
├── cctask
├── chat-e2e
├── logs
├── sql
├── workflow
├── check-dt-main-status.sh
├── claw-api-test.sh
├── hot_procs.sh
├── install_vibe_coding_cli.sh
├── multilang_batch_test.py
├── preview-slide.sh
├── slides-preview.sh
├── smux_demo.sh
├── start.sh
├── task-complete-hook.sh
├── test-env.sh
├── tmux_bridge_demo.py
├── tmux_claude_bridge_test.py
├── xkx-local-e2e-test.sh
├── xkx-local-env.sh
├── xkx-local-start.sh
├── xkx-task-worker.env.example
└── xkx-task-worker.sh
```

08 我是怎么做的（四） — Workflow Engine

三种执行模式

single — 全自动

1 个 AI 顺序执行 9 阶段
自动 check + next, 日常开发首选

manual — 半自动

每步人工确认 check + next
适合学习流程、精细控制

parallel — 多 Agent 协作

tmux 多窗格: analyst (主控) / coder / tester
只有主 Agent 能改状态, 权限隔离

文件锁并发安全 · 状态持久化 .workflow/state.json ·
reset 归档不删除

Audit Chain — AI 自己查自己

检查 → 修复 → 再验证, 最多 2 轮
文档完整性 / 代码评审(x2) / 端到端测试 / 表结构

互斥锁防并发

进程检测

超时保护

自动杀窗



每个阶段强制 Checkpoint — 文档完整性 / 结构规范 / 测试覆盖 / 代码规范
check 通过 → next / check 失败 → 修复后重检 / goto (跳过, 留审计记录) /
reset (归档不删除)

08 我是怎么做的（四） — 定时任务

cctask — 定时任务引擎

基于 `launchd` 每小时触发，AI 自动跑任务

- 11:00 — 自动 Code Review
- 16:00 — 自动 Code Review
- 18:00 — 自动生成日报

多仓库扫描，报告写入 `code-review-reports/`

07 我是怎么做的（三）—— 一个 Skill 的解剖

以 coding-standards 为例——从"每次都说一遍"到"说一句就全覆盖"

激活方式

说一句"按代码规范来", 7 个维度全部自动覆盖
不用每次说"注意日志""注意异常""注意性能"

SKILL.md — 主入口 (AI 自动加载)

定位 — 只要碰代码就按这个来

何时生效 — 写/改/重构/评审

7 维度总览 — 命名/外部调用/数据层/可观测/异常/性能/安全

自检清单 — 提交前逐条过

references/ — 按需加载 (不占上下文)

01 通用编码

02 外部调用

03 数据层

04 可观测性

05 异常处理

06 性能并发

07 安全编码

涉及 DB 操作 → 自动读 03; 涉及外部调用 → 自动读 02; 不涉及的不加载

上下游协作: system-design 设计 → coding-standards 落地 → code-review 检查

没有这个 Skill 时

- ✗ 每次都要说"注意日志""注意超时""注意异常"
- ✗ 说漏了 AI 就不做, 产出不稳定
- ✗ 同样的需求, 不同对话产出质量差异大
- ✗ 新人不知道要提醒 AI 哪些规范

有了这个 Skill 后

- ✓ 一句"按代码规范来", 7 维度全覆盖
- ✓ 涉及 DB 自动加载详细规范, 不涉及不浪费上下文
- ✓ 提交前自动走自检清单, 产出稳定
- ✓ 新人也能按同样标准产出代码

接下来 — 规划路线图

近期

夯实基础

- ▶ 数据开发与查询，运营产品介入
- ▶ 全员实践，前端测试深度使用
- ▶ 完善基础组件，补全整个链路
- ▶ 借助 Claude Code SDK 把自动化从 CLI 级升级到编程级稳定

中期

夜间无人值守

- ▶ 大需求夜间持续运行，白天人审核
- ▶ 生产环境监控与快速修复
- ▶ dt_robot 作为平台，非开发也能操作

远期

自进化

- ▶ 观察生产数据与问题，自动出需求
- ▶ 整套机制可复制到其他项目

从工具到平台，从辅助到 **自进化**

架构总览 — dt_robot 整体面貌



10 有哪些收益 — 提效最显著，数据收敛



提效

DeepTrip 更快地跑
需求交付周期缩短
重复劳动被自动化



文档沉淀

AI 持续消费文档，保持"活性"
新人 onboarding 成本降低
知识不再散落在人脑和聊天记录里



反向推动产品思考

AI 追问边界条件
倒逼产品把需求想清楚
间接提升文档质量



可以做更多尝试

以前觉得"太麻烦"的事，现在可以尝试
试错成本降低
创新的门槛降低了



释放想象力和创造力

想象力和创造力是核心竞争力
人从重复劳动中解放出来
专注"做什么"和"为什么做"



能力扩散

其他项目直接 fork dt_robot 拉分支
借助里面的规范约束直接开干
定期合并主分支能力即可
全程由 Claude Code 完成

dt_robot 带来的不只是效率，是 新的可能性

11 方法论分享 — AI 落地的七要素

正确的数据

数据要丰富、要准确
文档放在代码旁，Git 管控
AI 吃什么吐什么，垃圾进
垃圾出

好用的模型

用最好的：Claude
能力上限决定产出天花板
模型选错了，后面怎么补
都补不回来

好用的工具

Claude Code
降低使用门槛，让人人能
用
工具链打通了，效率才能
飞起来

正确的流程

弄清楚流程，用字写出来
不要每次口头说，每次说
得都不一样
流程标准化 = 可复制、可
审计

反复调教

20+ Skills 一版版迭代出来的，不是一
次写好
调教到相对稳定态，才能放心交给团队
用

必须亲自使用

真正用起来才会有体感
体感越深改进方向越准
光看不用 = 没感觉

人的经验与想象力

AI 再强也需要人的业务经验来把关
人的想象力来定义方向
七个要素，这是最不可替代的一个

// 七个缺一不可：AI 拿走了脑力，**心力和判断力**留给了我们

12 想法分享 — 一些实践中的思考

工具的进化

你的工具不适合模型来调用，你就需要做出改变

不做出改变，就会有新的工具来替换掉你

除非你有非常重的资源壁垒

但即使有壁垒，模型也会写其他软件来方便自己调用

让模型能轻松调用，是工具设计的新标准

终端的力量

模型 + Unix 能力 + 最轻量的表现形态

目前所有交互都在终端——这是最好的方式

最纯粹的人机交互，没有界面的干扰

直接与系统对话，组合能力无限

= 爆发出最强的能量

自动化流程

从"人执行每一步"到"人把控方向，AI 执行"

六个基础设施：知识 · 规范 · 流程 · 脚本 · 兜底机制 · 可视化

缺一个，自动化就跑不稳

可观测性

AI 做了什么，需要能看到

日志、trace、产出物都要可追溯

人要能判断 AI 的产出对不对

可观测性非常重要

页面/UI 不是最重要的——最终形态千人千面，只会留下一些真正有创意的想法和设计，才能永久保留

13 延伸思考 — 两期让我受益的播客



人何以自处

AI 时代人的定位

"工业革命拿走了体力，AI 正在拿走脑力。留给人的，是心力。"

"Your feed is your fate"——信息输入塑造人生

"不是你知道什么，而是你想要什么。不是你能想出什么，而是你能感受到什么。"

"品味即权重：审美，是个人神经元被反复冲刷后的显影"



OpenAI 翁家翌访谈

成长历程与技术洞察

"开源不是履历装饰，而是对世界的投入方式：追求的不是掌声，而是 impact"

"人生在于主动选择或构建自己的评价体系"



Q & A

欢迎提问和讨论

思路碰撞

实践交流

一起探索

DeepTrip Team · 徐凯旋 · 2026.04.22